Who This Book Is For

This book was written for practicing Python programmers who want to become proficient in Python 3. I tested the examples in Python 3.10—most of them also in Python 3.9 and 3.8. When an example requires Python 3.10, it should be clearly marked.

If you are not sure whether you know enough Python to follow along, review the topics of the official Python tutorial. Topics covered in the tutorial will not be explained here, except for some features that are new.

Who This Book Is Not For

If you are just learning Python, this book is going to be hard to follow. Not only that, if you read it too early in your Python journey, it may give you the impression that every Python script should leverage special methods and metaprogramming tricks. Premature abstraction is as bad as premature optimization.

Five Books in One

I recommend that everyone read Chapter 1, "The Python Data Model". The core audience for this book should not have trouble jumping directly to any part in this book after Chapter 1, but often I assume you've read preceding chapters in each specific part. Think of Parts I through V as books within the book.

I tried to emphasize using what is available before discussing how to build your own. For example, in Part I, Chapter 2 covers sequence types that are ready to use, including some that don't get a lot of attention, like `collections.deque`. Building user-defined sequences is only addressed in Part III, where we also see how to leverage the abstract base classes (ABCs) from `collections.abc`. Creating your own ABCs is discussed even later in Part III, because I believe it's important to be comfortable using an ABC before writing your own.

This approach has a few advantages. First, knowing what is ready to use can save you from reinventing the wheel. We use existing collection classes more often than we implement our own, and we can give more attention to the advanced usage of available tools by deferring the discussion on how to create new ones. We are also more likely to inherit from existing ABCs than to create a new ABC from scratch. And finally, I believe it is easier to understand the abstractions after you've seen them in action.

The downside of this strategy is the forward references scattered throughout the chapters. I hope these will be easier to tolerate now that you know why I chose this path.

How the Book Is Organized

Here are the main topics in each part of the book:

Part I, "Data Structures"

Chapter 1 introduces the Python Data Model and explains why the special methods (e.g., `__repr__`) are the key to the consistent behavior of objects of all types. Special methods are covered in more detail throughout the book. The remaining chapters in this part cover the use of collection types: sequences, mappings, and sets, as well as the `str` versus `bytes` split—the cause of much celebration among Python 3 users and much pain for Python 2 users migrating their codebases. Also covered are the high-level class builders in the standard library: named tuple factories and the `@dataclass` decorator. Pattern matching—new in Python 3.10—is covered in sections in Chapters 2, 3, and 5, which discuss sequence patterns, mapping patterns, and class patterns. The last chapter in Part I is about the life cycle of objects: references, mutability, and garbage collection.

Part II, "Functions as Objects"

Here we talk about functions as first-class objects in the language: what that means, how it affects some popular design patterns, and how to implement function decorators by leveraging closures. Also covered here is the general concept of callables in Python, function attributes, introspection, parameter annotations, and the new `nonlocal` declaration in Python 3. Chapter 8 introduces the major new topic of type hints in function signatures.

Part III, "Classes and Protocols"

Now the focus is on building classes "by hand"—as opposed to using the class builders covered in Chapter 5. Like any Object-Oriented (OO) language, Python has its particular set of features that may or may not be present in the language in which you and I learned class-based programming. The chapters explain how to build your own collections, abstract base classes (ABCs), and protocols, as well as how to cope with multiple inheritance, and how to implement operator overloading—when that makes sense. Chapter 15 continues the coverage of type hints.

Part IV, "Control Flow"

Covered in this part are the language constructs and libraries that go beyond traditional control flow with conditionals, loops, and subroutines. We start with generators, then visit context managers and coroutines, including the challenging but powerful new `yield from` syntax. Chapter 18 includes a significant example using pattern matching in a simple but functional language interpreter. Chapter 19, "Concurrency Models in Python" is a new chapter presenting an overview of alternatives for concurrent and parallel processing in Python, their limitations, and how software architecture allows Python to operate at web scale. I rewrote